Heidelberg University
Institute of Computer Science
Artificial Intelligence for Programming (AIP)

Master's Thesis

Interrupt-Guided Neural Code Completion with Static Analysis of Dependencies

Name:	Niklas Loeser
Matrikelnummer:	3737339
Betreuer:	Prof. Dr. Artur Andrzejak°
Datum der Abgabe	2024-09-30

I hereby certify that I have written the work myself and that I have not used any sources or aids other than those specified and that I have marked what has been taken over from other people's works, either verbatim or in terms of content, as foreign. I also certify that the electronic version of my thesis transmitted completely corresponds in content and wording to the printed version. I agree that this electronic version is being checked for plagiarism at the university using plagiarism software.

Heidelberg, the 2024-07-06

Niklas Loeser

Abstract

- English -

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et aut officiis debitis aut rerum necessitatibus saepe eveniet, ut et voluptates repudiandae sint et molestiae non recusandae. Itaque earum rerum defuturum, quas natura non depravata desiderat. Et quem ad me accedis, saluto: 'chaere,' inquam, 'Tite!' lictores, turma omnis chorusque: 'chaere, Tite!' hinc hostis mi Albucius, hinc inimicus. Sed iure Mucius. Ego autem mirari satis non queo unde hoc sit tam insolens domesticarum rerum fastidium. Non est omnino hic docendi locus; sed ita prorsus existimo, neque eum Torquatum, qui hoc primus cognomen invenerit, aut torquem illum hosti detraxisse, ut aliquam ex eo est consecutus? – Laudem et caritatem, quae sunt vitae sine metu degendae praesidia firmissima. – Filium morte multavit. – Si sine causa, nollem me ab eo delectari, quod ista Platonis, Aristoteli, Theophrasti orationis ornamenta neglexerit. Nam illud quidem physici, credere aliquid esse minimum, quod profecto numquam putavisset, si a Polyaeno, familiari suo, geometrica discere maluisset quam illum etiam ipsum dedocere. Sol Democrito magnus videtur, quippe homini erudito in geometriaque perfecto, huic pedalis fortasse; tantum enim esse omnino in nostris poetis aut inertissimae segnitiae est aut fastidii delicatissimi. Mihi quidem videtur, inermis ac nudus est. Tollit definitiones, nihil de dividendo ac partiendo docet, non quo ignorare vos arbitrer, sed ut.

Abstract

- German -

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et aut officiis debitis aut rerum necessitatibus saepe eveniet, ut et voluptates repudiandae sint et molestiae non recusandae. Itaque earum rerum defuturum, quas natura non depravata desiderat. Et quem ad me accedis, saluto: 'chaere,' inquam, 'Tite!' lictores, turma omnis chorusque: 'chaere, Tite!' hinc hostis mi Albucius, hinc inimicus. Sed iure Mucius. Ego autem mirari satis non queo unde hoc sit tam insolens domesticarum rerum fastidium. Non est omnino hic docendi locus; sed ita prorsus existimo, neque eum Torquatum, qui hoc primus cognomen invenerit, aut torquem illum hosti detraxisse, ut aliquam ex eo est consecutus? – Laudem et caritatem, quae sunt vitae sine metu degendae praesidia firmissima. – Filium morte multavit. – Si sine causa, nollem me ab eo delectari, quod ista Platonis, Aristoteli, Theophrasti orationis ornamenta neglexerit. Nam illud quidem physici, credere aliquid esse minimum, quod profecto numquam putavisset, si a Polyaeno, familiari suo, geometrica discere maluisset quam illum etiam ipsum dedocere. Sol Democrito magnus videtur, quippe homini erudito in geometriaque perfecto, huic pedalis fortasse; tantum enim esse omnino in nostris poetis aut inertissimae segnitiae est aut fastidii delicatissimi. Mihi quidem videtur, inermis ac nudus est. Tollit definitiones, nihil de dividendo ac partiendo docet, non quo ignorare vos arbitrer, sed ut.

Contents

1 Introduction 1
1.1 Motivation 1
1.2 Goals 2
1.3 Contributions 2

2 Background & Related Work 4
2.1 Causal Language Models 4
2.2 Text Generation Strategies 6
2.3 Controllable Generation 7
2.4 Torch 10
2.5 Transformers Library 12
2.6 Language Server Protocol 17
2.7 Related Work 25

3 Design 26
3.1 Requirements 26
3.2 Analysis 28
3.3 Interrupt-based meta-strategy 29
3.4 Language Server Protocol aided generation 33

4 Implementation 37
4.1 Interrupt-based meta-strategy 37
4.2 Language Server Protocol aided generation 37
4.3 Dataset design 37

5 Results 38
5.1 Collection method 38
5.2 Possible configurations 38
5.3 Results 38

6 Evaluation 39
6.1 Research Questions 39
6.2 Evaluation difficulties 39
6.3 Evaluation 39

7 Conclusion 40
7.1 Recap 40
7.2 Outlook 40

Acronyms 41

Bibliography 42

Listings 45
List of Figures 45
List of Tables 45
List of Code 45
List of Examples 46

Appendix 47

1 Introduction

Code completion provides code suggestions to developers extending their incomplete code. The suggestions used multimodal data sources to combine a variety of information.

It is one of the most used productivity tools in Integrated Development Environments (IDEs). Every major IDE from VSCode, Jetbrains IDEs, Eclipse, Neovim and Emacs with plugins support code completion. It reduces the mental burden of remembering method names, properties, their types and associated information, such as documentation and if the item is deprecated. In addition it saves time by reducing written boilerplate code by suggesting code snippets. Using the code completion saves time spent searching for corresponding documentation outside of the IDEs.

As Amann et al.[1] found, code completion is the most frequently used tool inside IDEs with a frequency of 19.7%. Moreover, developers spend 22.4% of their time inside IDEs navigating code, in contrast to 28.5% editing and executing the code. They hypothesize that a significant amount of time navigating code is spent due to a lack of understanding of the code. Moreover, 39.8% of the total time is spent outside the IDE, which likely includes time spent searching for documentation.

Good code completion is a crucial tool for developers, reducing time spent in total and reducing the time spent outside the IDE, further increasing the focus of the developer.

1.1 Motivation

Neural code completion tools, such as GitHub Copilot, have seen a massive adoption in the software development community.

Such tools promise to complement typical code completion tools by providing context-aware code completions spanning multiple lines. These multi-line completions reduce the need to write boilerplate code further, thus improving the productivity.

While typical code completion tools use static analysis to provide completions, neural code completions tools use causal language models trained on code to predict the next tokens. As Ziegler et al. [2] have shown, the completions provided by Copilot serve as a template to tinker with.

Even if the completions provide a starting point, this work argues that current neural code completions do not use enough context to be accurate and are insufficient if used without the correct documentation.

Causal Language Models (CLMs) are released together with a "cutoff date", which is the date until which the training data has been collected. These models are incapable of predicting code completions which rely on information released after the cutoff date. Even worse, the training data may contain information on a mixture of several old versions of the same item.

Widely used code packages are updated frequently, in which code is added, removed, or changed. Often the code is not removed directly, but marked as deprecated. It is still available, but should not be used anymore.

It is no surprise, that given a coding environment with specific versions of packages, the completions are often incapable of using correct code items. When using cutting-edge packages, the suggestions may not know anything, contain now renamed function parameters or use long deprecated methods. For legacy projects the reverse is true, as the neural code completion may suggest items from newer versions, which are not available in the project. As such, developers are forced to use the suggestions as a rough template, which they have to adjust to the correct documentation. Hence, developers use classical IDE tooling or consult documentation directly to check single suggested items.

Methods such as retraining the model with correct package information ad-hoc are not feasible, as they are slow and require a lot of computational resources. Other dynamic

methods providing the model with up-to-date information are not flexible enough and do not provide enough context to be accurate yet. Additionally, some current methods are too slow to be used in a real-time coding environment.

1.2 Goals

This thesis aims to combine up-to-date information through static analysis with the prediction capabilities of CLMs. The combination shall provide the neural code completion with the contextual information needed, to suggest code fitting the local code environment. The contextual information shall be provided at specific instances to guide the model **away** from incorrect suggestions **towards** correct suggestions. Additionally, the performance impact of the proposed approach shall be kept low, to ensure a responsive system. This will enable intregration into IDEs and other tools.

Initially, the thesis will introduce important concepts and technologies, from Causal Language Modeling to Controllable Generation, followed by implementation specific libraries. Moreover, related work will be introduced to provide a basis for the proposed approach.

Afterwards, the thesis will introduce the requirements for a proposed approach, then analyze shortcomings of current approaches and propose a new approach to address these shortcomings. Possible solutions will be discussed and evaluated, to find a promising approach.

Then, the thesis will describe the components of the proposed approach.

Finally, the approach will be implemented and evaluated, according to the defined requirements.

At last, the thesis will conclude with a summary of the results and an outlook on future work.

1.3 Contributions

The main contributions of the thesis are:

1. Performant, robust and generalizable neural code completion using up-to-date package information.
2. A novel Interrupt-based meta-strategy (IBMS) for injecting context into the model.
3. A dataset for evaluation functional and approach correctness.

The main contribution integrates information from the currently used packages and local repository into the code completion. The information is retrieved through the code completion and signature help functionality, provided by typical language servers, which generalize this work over different programming languages. The information includes deprecation information, preventing the model from completing old methods and guiding it towards replacements, using the provided documentation. Performance wise, the framework only incurs a small performance penalty over the usual predicted code completion.

The second contribution is a novel IBMS, which may be used outside of code completion. It enables monitoring the prediction similar to Agrawal et al.[3], but then injects retrieved information at critical timepoints into the context “during” the generation. It is used to inject the retrieved information into the model, nudging it towards the correct completion.

Lastly, the dataset evaluates, if the completed code is functionally correct and uses the correct approach. The evaluation checks, if e.g. a deprecated method was used and then fails the completed code. The dataset includes hand-picked code pieces of rapidly changing packages, legacy packages or new and unused packages. Each item includes the Python version and pinned dependencies, to create a reproducable code environment. To

secure the evaluation, the evaluation pipeline uses Docker containers. In addition, the evaluation generates a report and draws diagrams.

2 Background & Related Work

This chapter provides important background information and related work, necessary to understand the main contributions of this thesis. Firstly, CLMs are introduced in Section 2.1. Thereafter, Section 2.2 discusses commonly used text generation strategies.

The section Section 2.3 showcases strategies to modify the text generation of a fixed pretrained CLM. Section 2.5 displays important components of the popular Hugging Face libary, which enables CLM workflows using all of the techniques introduced in the sections before.

Thereafter, Section 2.6 introduces the functionality of the Language Server Protocol (LSP). At last, Section 2.7 introduces previous work.

2.1 Causal Language Models

This section will first introduce the concept of a Language Models (LMs), then move from neural language models, to CLMs. Important text generation strategies for CLM are discussed in depth in the next section.

Language Models

Generally, LMs are probabilistic models of natural language[4]. They predict the next upcoming words based on already seen words[4]:

Previous text	*"The quick brown fox jumps over the lazy _"*
Prediction	*"dog"*

Example 1: Language model predicting the last word of the famous English pangram.

Development on LMs has led to powerful models. These now find widespread use in various applications, such as ChatGPT[5].

Neural Language Models

Neural language models are a type of LM. Their name stems from the McCulloch-Pitts neuron, which is a simplified view of a biological neuron as a computational unit[6], [4].

In contrast, a modern neural language model is a network of small computing units. Each of the units is a function that takes a sequence of values and returns a singular output value[4]:

$$f_{\text{unit}} : \mathbb{R}^n \to \mathbb{R} \tag{1}$$

Combining many units into a network allows the computation of a complex function f_{network}. The behavior of f_{network} can be changed by modifying the parameters inside of the network. By applying changes according to feedback, complex functions can be learned.

One very popular architecture for neural language models is the Transformers architecture.

As can be seen in Equation 1, the network is unable to process sequences of natural language directly. The text must first be encoded into numbers, the model can work with. After processing the encoded text, the result can be decoded into natural language once again.

Each neural language model encodes natural language into increasing numbers called tokens using a vocabulary V. The vocabulary is usually fixed, but can be changed as will be shown in the thesis. Each token represents a piece of natural language, such as a part of a word, or even parts of a sentence. The models initially map natural language

to tokens for processing. After processing the tokens these are decoded back into natural language.

Vocabulary	*{"The", "quick", "brown", "fox", "jumps", "over", "the", "lazy", "dog", "_", ".", "<BOS>", "<EOS>"}*
Text	*"The quick brown fox jumps"*
Tokens ids	*{11, 0, 9, 1, 9, 2, 9, 3, 4}*
Tokens	*{"<BOS>", "The", "_", "quick", "_", "brown", "_", "fox", "_", "jumps"}*

Example 2: Natural language encoded into tokens.

Example 2 showcases the tokenization for the start of the famous English pangram. The tokens values correspond to the index in the given vocabulary. The given vocabulary only provides tokens for full words and does not provide tokens for sub-word fragments. Not only does the vocabulary contain tokens for natural language, but also special tokens. These special tokens encode further semantic information for the CLM. Common special tokens are the Begin of Sequence (BOS), End of Sequence (EOS) and Padding (PAD) tokens:

BOS Marks the beginning of a natural text. Used to delimit several unrelated texts.

EOS Marks the end of a natural text. Used in combination with BOS. The text generation is stopped, when the model predicts the EOS token as the next token.

PAD Used to pad sequences from either side to a fixed length. This is necessary for the model to process several sequences of varying lengths at once.

Definition

Causal language models in turn are a type of neural language model. They rely on the assumption that the probability of a word occuring in a document depends only on the words occuring before it.

This property is called autoregressivity and highly favourable towards text generation, as it enables the continuation of any text. Given a text i_0 as input, the causal language model can predict the next word, as all previous words i_0 are known. After predicting the next word, the word is appended to the input to form the new input i_1. This process is repeated until the model predicts the end of the document using the special EOS token after n steps. i_n then forms the output and the whole generated text.

When applied to text completion or code completion, a program need only return the output i_n without the prefix i_0.

For the purposes of this thesis, a CLM can be defined using the following components:

- Raw next token prediction function f_{rntpf}
- Text generation strategy
- Autoregressive property

The raw next token prediction function is a mathematical function that takes a sequence of tokens and returns the prediction for the next tokens:

$$\begin{aligned} f_{\text{rntpf}}(x) &= u \\ f_{\text{rntpf}} &: \mathbb{N}^n \to \mathbb{R}^{|V|} \end{aligned} \tag{2}$$

The output u has the shape of the vocabulary, where each value u_i represents the logistical probability of token i being the next token. These values are called logits.

In an additional step, the logistical probabilities are normalized into values in the range of $(0, 1)$ to form a probability distribution.

The intermediary step of forming logits provides the benefit, that individual logits can be changed individually without disrupting the probability distribution. This property is essential for the main contributions in this thesis.

2.2 Text Generation Strategies

To generate a full sequence of tokens and thus a document, the model must choose the next probable token in the document using a text generation strategy TGS. After choosing the next token $\hat{w}$, where $1 \leq \hat{w} \leq |\text{vocab}|$, it is appended to the old input x to form x':

$$x' = x^\frown\langle\hat{w}\rangle = (x_1, x_2, ..., x_n, \hat{w}) \tag{3}$$

. Depending on the text generation strategy, the models generate different documents of varying quality. Each text generation strategy has different advantages and drawbacks. The next sections will introduce a few core sampling strategies and highlight the main advantages and drawbacks.

Greedy Decoding

Greedy decoding is the simplest of sampling strategies. The strategy always chooses the token with highest probability. This always yields the current best next token, but not necessarily the best token overall.

Its main advantage is the simplicity. The strategy does not require additional state. Only the current probabilities are needed. This also leads to low hardware requirements, compared to other strategies.

As the current best next token is not often the best token overall, the strategy does not produce the highest quality documents.

$$\hat{w} = \text{argmax}_{w \in V} P(w \mid w_{<n}) \tag{4}$$

Beam Search

Greedy decoding suffers from the problem of always choosing the current most probable token, even if it is not the best token overall. Beam search improves upon this by modeling the decoding process as a search tree. Instead of choosing only a single token, the strategy keeps k possible token choices. These k possible token choices are called hypothesis. At each time step the beam search uses the current k hypothesis to independently continue the generation to create $k * V$ possible token combinations. The strategy then keeps the top-k combinations as new hypothesis.

By modeling the decoding process as a search tree, beam search can choose tokens that are better overall. This leads to a higher quality generation.

But as beam search keeps k hypothesis and decodes $k * V$ possible token combinations, the strategy requires more memory and processing power.

Top-k

Top-k is a generalization of greedy decoding and a sampling strategy. Sampling strategies randomly choose the next token from the probability distribution. Top-k chooses the next token from the top k most probable next tokens. Hence, the model chooses a wider variety of tokens, leading to richer outputs.

As top-k is a generalization of greedy decoding, it has the advantage of being more powerful.

The main drawback is the loss of determinism. Lacking determinism may complicate interpretability methods.

Top-p

Top-k always chooses from the top k most probable tokens, even if most tokens are very unlikely and only few of the k tokens are probable. Top-p allows for more dynamic shapes of probability distributions, by only sampling from the top p percent of tokens.

Temperature sampling

Temperature sampling takes a different approach to top-p. It does not adapt to the shape of the probability distribution, but reshapes it. It divides the raw probability scores by a temperature τ:

$$y = \text{softmax}\left(\frac{u}{\tau}\right) \tag{5}$$

The softmax function normalizes the logits to the range $(0, 1)$, where the sum of all values equals 1. When dividing by $\tau < 1$ all values increase in total. After normalization this skews the probability distribution towards the most probable tokens. In contrast, when dividing by $\tau > 1$ the probability distribution is flattened. This leads to a more uniform distribution.

2.3 Controllable Generation

CLMs suffer from many problems, such as outdated knowledge, insufficient knowledge and insufficient capabilities. Most of these problems can be adressed by fine-tuning the model on further data, such as up-to-date information. The problem with fine-tuning is the need for a large amount of up-to-date data, which might not exist yet. In addition. fine-tuning is resource intensive and must be performed continously, to stay up-to-date.

This section will showcase different ways to control the generation of the model, without the need to perform fine-tuning. The text generation is complex and there are many different strategies and components, which may be controlled. Guided generation will focus in strategies to control the prediction of a single next token, while the chapter prompting strategies will focus on prompting techniques.

Guided Generation

The CLM generates a text, by iteratively choosing the next token ids. The decision on which token ids is the next token id, depends on two factors, as discussed in Section 2.1. It depends on the logits produced through the raw next token prediction function f_{rntpf} and a decoding strategy. Guided generation is a technique to modify the logits before the decoding strategy is applied. The logits are modified according to a set of domain specific rules. The rules not only have access to the current logits, but also to the previously generated text. Thes allows for complex rules.

Consider the following example:

A model is tasked with answering a “yes” or “no” question. The model should only respond with either of both words, without discussing any details or making small talk.
The rules for the guided generation are to set the logits of all tokens to $-\infty$, except when the tokens contribute to a “yes” or “no” answer.
Given the prompt “Is the sun shining?” and the previous tokens “Ye”, the guided generation will set all logits to $-\infty$, except for the token “s”. In the next step only the EOS token will be permitted. Thus, the model will respond with “Yes”.

Example 3: Enforcing "yes" or "no" answers

Guided generation has the benefit of allowing complex rules to be applied to the generation process, to allow a very tightly controlled generation.

The downside of guided generation is, that for complex scenarios, the rules may become very complex and hard to define. In these cases prompting strategies might be more appropriate.

[7]

Prompting strategies

Prompting strategies are a set of techniques to control the generation of the model externally. The model itself, or the model runtime itself is not modified. As such this technique is model agnostic. The general goal is to control the output through structured prompts. In addition, the model may be incorporated into external tooling to enable complex workflows.

The simplest form of control through prompting is to add instructions to the prompt. Models will follow the instructions, if the model was trained on instructions in a similar format. So called "instruction fine-tuned models" were fine-tuned especially on instructions, to improve instruction following capabilities. Non instruction fine-tuned models can still follow instructions, but then the instructions must be chosen carefully.

A useful instruction is to define the desired output format. For example, by specifying that the model should output the response in JavaScript Object Notation (JSON) format or Markdown format, the model will use the specified format. Especially formats such as Markdown and JSON are preferred, as they are common formats and thus the model performs especially well in these formats. Specifying the desired format increases the predictability of the response, which allows automated processing of the response. Specifying to respond in an unknown format does not work. Steering the model does not raise the capabilities of the model, but raises the efficiency with which its capabilities are used.

If the model does not have enough knowledge to respond to a prompt, the prompt itself can be structured to supply the needed information. Given the model has the ability to understand the provided information, it can then act on the information. A popular technique is **few-shot prompting**[8].

Few-shot prompting involves providing the model with several examples in the prompt. The goal is that the model understands the provided examples to learn the pattern and apply it to its response. The learning is undertaken "in context" and not persisted outside of the prompt itself[8]. The knowledge provided to the model is the format of the task and matching response.

Prompt:
Q: What is the capital of Germany?
A: Berlin
Q: What is the capital of France?
A: Paris
Q: What is the capital of Italy?

Response:
A: Rome

Example 4: Few-shot prompting

As can be seen in Example 4, the model prefixes the response with A: and does not provide additional information. The output format was not specified explicitly, but matches the format of the examples perfectly.

In contrast to providing knowledge about a desired response format, **context injection** adds important context into the prompt, which the model then uses to respond.

Prompt:
The capital of Germany was changed to Heidelberg.
What is the capital of Germany?

Response:
Heidelberg

Example 5: Context injection

Example 5 demonstrates how adding information controls the response of the model. This technique is very powerful, but comes with two strong restrictions:
1. Finding relevant context.
2. Limited prompt size.

The first restriction is that the injected context must first be found. This is a hard search problem, as the information must be retrieved **before** prompting, before the intent of the prompt is known yet. In Example 5 the task of the CLM is question answering. Even though the general task is known, the software has no way to known the specific domain, such as "country capitals" beforehand.

The second restriction is the limited prompt size. Models are only capable of focusing on a prompt of limited size. Thus, token space is precious and may not be wasted. Providing the model with an up-to-date version of Wikipedia for every prompt oversaturates the prompt size limit instantly. The injected context must be **targeted** and **relevant**. The more **targeted** the context is, the less tokens are used.

As finding relevant context is a hard problem, several steps are needed to search for the context. Searching for relevant context may even involve the output of another CLM. Strategies that involve the model in a multi-step process are called meta-strategies. These strategies will be further discussed in the next section.

Meta-Strategies

Meta-strategies incorporate a model into an algorithm according to a wider strategy. These strategies shift the focus to viewing the CLMs as a building block to be used as needed. Several models with different strenghts could be combined into a meta-strategy.

Generally, the main benefits are ease of use and flexibility. Any model can easily be integrated into the meta-strategy and stay interchangable. By combining several models into a multi-turn process, including external tooling, powerful behavior is possible.

In contrast, common drawbacks of meta-strategies are the added complexity towards the already complex text generation. In addition, calling models several times increases the runtime of the program, if used without care.

Meta-strategies are applied to e.g. remedy the model lack of up-to-date knowledge. A model may not have the necessary knowledge for a task, or the knowledge it has is outdated. In such cases the meta-strategy can supply up-to-date information and focus the model towards the problem.

Chain of Thought Chain of Thought is one very central meta-strategy, as it significantly improves the reasoning capabilities of a model[9].

The key idea of chain of thought is to apply the divide and conquer strategy to the text generation process. Instead of the model trying to solve a complex reasoning problem in a single step, the meta-strategy divides the problem into several intermediate step. The model is then able to solve the simpler intermediate steps. Each intermediate step is a thought the model contributes to the final solution. The sequence of intermediate steps forms the "chain" of thoughts[9].

Additionally, the model will use thoughts of previous steps to reason about the next step.

Tree of Thought[1] Tree of thoughts is a generalization of chain of thought. Chain of thought is only capable of performing linear decision-making[9]. Ultimately, performing complex reasoning is a search problem and thus, linear reasoning is insufficient. The tree of thoughts meta-strategy generalizes the thoughts into a tree structure. Each intermediate problem is then a node in this search tree. The root of the tree is the initial problem. The goal of this search problem then is to find an accepted end node - a finishing thought[10].

By structuring the thoughts into a tree, common tree search algorithms can be applied to the problem. Various algorithms such as a breadth-first search or depth-first search can be applied[10].

In general, the tree of thoughts meta-strategy is more powerful than the chain of thoughts. It is a very powerful and flexible strategy, which even allows following thoughts and then backtracking[10].

[10]

2.4 Torch

Torch is a widely used machine learning library in Python. It enables the creation and manipulation of tensors in different precisions. The Transformers provides high level abstractions which base on Pytorch. This section shortly introduces a few core concepts of Pytorch, which are used in the Transformers library. It will focus on version 2.2° of PyTorch.

PyTorch provides operations on tensors. A tensor is a multi-dimensional array of numerical values and the generalization of a matrix[11]. Each tensor typically only contains values of the same type, such as value from $\mathbb{R}$[11].

One of the applications for tensors is to represent multi-relational data[11]. In that case each dimension represents a single relation.

For CLM inference different types of tensors are used. For example, the token sequence during parallel text generation is a two-dimensional tensors of the following relations: (batch, token). The second dimension represents a single token sequence, while the first dimension enables parallel text generation for n token sequences.

Another example is the next token prediction tensor, which is a two-dimensional tensor of the following relations: (batch, vocabulary). The second dimension contains the probability distribution over the whole vocabulary, while the first dimension enables parallel text generation.

The second example only displays the tensor for a single next token prediction. By adding another dimension for the token relation, the whole probability distribution history could be represented.

After providing the motivation for tensors, the following list will introduce core terminology for use with tensors:

Order Number of dimensions of a tensor
Shape Number of elements in each dimension
Slice Two-dimensional subfield of a tensor. Obtained by fixing all but two indices.
Fiber One-dimensional subarray of a tensor. Obtained by fixing all but one index.
Matrix Two-dimensional tensor.
Vector One-dimensional tensor.
Scalar Zero-dimensional tensor.

[11]

[1] **Note:** Can be removed with Chain of thought

Fixing indices on a tensor will return a subfield or subarray, which in itself is a lower-order tensor. By fixing all indices a scalar is obtained. A slice and fiber are special terms for lower-order tensors obtained from fixing indices on a tensor.

As introduced earlies, tensors usually contain values of the same type. Each PyTorch tensor contains values of a single data type, in short: dtype. PyTorch supports many different numerical data types, of which the following are relevant:

Data Type	dtype
32-bit floating point	torch.float32
16-bit floating point 1	torch.float16
16-bit floating point 2	torch.bfloat16
64-bit integer (signed)	torch.int64
32-bit integer (signed)	torch.int32
Boolean	torch.bool

Table 1: Relevant PyTorch data types[12]

The data types can be used by specifying the dtype parameter when creating a tensor. The value is one of the variables listed in the dtype column in Table 1[13].

The difference between torch.float16 and torch.bfloat16 is the precision of the floating point number. Both formats use 16 bits to encode the floating point number. The difference lies in the number of bits used to encode the mantissa[12].

The torch.float16 is a half-precision floating point number, as defined in the IEEE 754 standard. It uses one sign bit, five exponent bits and 10 mantissa bits[14].

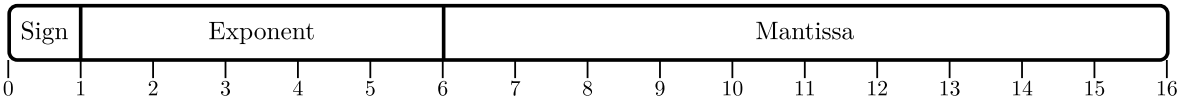


Figure 1: IEEE 754 half-precision floating point number: float16

In contast, the torch.bfloat16 keeps the full range of the full-precision floating point number of 8-bits, but has a reduced precision of 7 mantissa bits[15], [12].

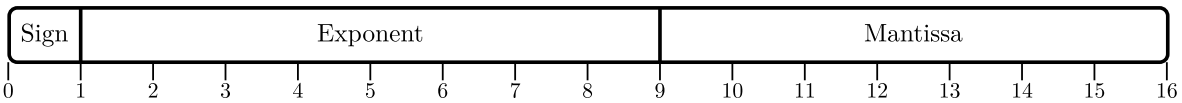


Figure 2: Brain Floating Point Format: bfloat16

The format was introduced by Google Brain and is used in their Tensor Processing Units (TPUs)[15], as the reduced mantissa precision reduces the size needed for physical hardware multipliers[15].

In addition to the precision, another aspect of a PyTorch tensor is the **physical location** it is allocated at, the device. The function torch.device creates a reference to such a physical location. Possible relevant device locations are cpu and cuda, where cpu allocates a tensor on the CPU memory, while cuda allocates the tensor on GPU memory[12]. Already creates tensor can be **moved to a different device** by calling the to method with the new device[16].

To **create a tensor**, the function torch.tensor is used. It takes data and optionally dtype and the device[13]. The order and shape of the tensor can be accessed through the dim and shape attributes[17], [18]. Indexing the tensor is done through the Python tensor[indice_list] notation, where untouched dimensions are marked through the : symbol, e.g.: tensor[3,:,1][19]. Iterating the tensor will iterate over the first dimension.

2.5 Transformers Library

Transformers is the name of the popular Python machine learning library developed by HuggingFace. It provides broad support for many different machine learning domains, such as computer vision, audio processing, multimodal models and **natural language processing**[20]. For each domain, the library enables simple training and inference of state-of-the-art models. The thesis heavily relies on functionality provided by this library, as it provides in-depth features to implement the more advanced features. It provides support for all techniques introduced in Section 2.1 and Section 2.2[20].

As the library receives frequent updates, the thesis focuses on version 4.41.3°. In addition, the library is complex and enables many different use-cases through many abstraction layers. This section will focus on the CLM use-case, omit unneeded arguments and list parameters only available for CLM.

The structure of this section will mirror the structure of previous sections. It will start by introducing CLMs, then continue with tokenizers. Afterwards, the text generation functions are presented.

Using different text generation strategies with the Transformers library will follow. At last, implementing controllable generation will be shown.

Models

In transformers, every type of model architecture is represented by a model class. Each model defines the layers of the model and the operations between layers. Common model functionality is inherited through the base class PreTrainedModel[21].

Pretrained models provide the following functionality (not exhaustive)[21]:

- Loading pre-trained weights
- Resizing the token embeddings

Loading a pre-trained model for inference is done by calling the from_pretrained method on the model class:

```python
from_pretrained(
    pretrained_model_name_or_path,
    torch_dtype: str | torch.dtype = None,
    device_map: str | Dict[str, Union[int, str, torch.device]] = None,
    **kwargs
) -> PreTrainedModel
```

Listing 1: Loading a pre-trained model[22]

Calling Listing 1 returns a model instance with pre-trained weights, loaded onto one or several physical devices. The model is either loaded from a local directory, or from the HuggingFace model hub. Where the model is loaded from depends on the type of the pretrained_model_name_or_path parameter.

The HuggingFace model hub allows for easy access of pre-trained models, given the name of the model and the name of the publisher. The model name has the format publisher_name/model_name. Some[2] of the many available CLM for code generation are:

- codellama/CodeLlama-7b-Instruct-hf
- deepseek-ai/deepseek-coder-6.7b-instruct
- m-a-p/OpenCodeInterpreter-DS-6.7B
- ise-uiuc/Magicoder-S-DS-6.7B

In the case that a model name from the HuggingFace hub is provided, the model is downloaded and cached locally. If the model is already present, it is loaded from cache.

[2]More code generation models can be found in the Bigcode models leaderboard°

By default, the model will be loaded using the data type of the loaded weights. The torch_dtype parameters allows loading the weights in e.g. lower precision, to save memory during inference.

The device_map parameter controls on which devices the model is allocated. The parameter takes a PyTorch device as value. Passing auto computes the most optimized device map automatically.

Transformers provides an abstraction called AutoClasses. These abstraction solve the problem of finding the specific model, config and tokenizer classes to load and use a CLM. Instead, the correct classes can be guessed from the name of a model alone[22]. This simplifies code when trying different models for the same task, such as code generation:

```python
from transformers import AutoModel, LlamaForCausalLM
# The following line
model = LlamaForCausalLM.from_pretrained("codellama/CodeLlama-7b-Instruct-hf")
# Would be transformed into
model = AutoModel.from_pretrained("codellama/CodeLlama-7b-Instruct-hf")
```

Listing 2: Using AutoClasses to load a model[22]

The method resize_token_embeddings resizes the token embeddings of the model to a new size. The function is commonly used to account for new special tokens added to the tokenizer at runtime[21].

Tokenizers

Tokenizers provide the functionality to convert text to and from sequences of token ids. In addition, they store the vocabulary, including information on special tokens, such as the EOS token[23].

Loading a tokenizer is done through the AutoClasses abstraction, which will return an instance of a subclass of a PreTrainedTokenizer:

```python
from transformers import AutoTokenizer
tokenizer = AutoTokenizer.from_pretrained("codellama/CodeLlama-7b-Instruct-hf")
```

Listing 3: Loading a tokenizer[22]

The vocabulary can be accessed through the get_vocab method, which returns a dictionary mapping tokens to token ids[23]. Special tokens and token ids can be accessed directly through special attributes of the tokenizer:

Token attribute	Token ID attribute	Description
bos_token	bos_token_id	Beginning of sequence token and token id
eos_token	eos_token_id	End of sequence token and token id
pad_token	pad_token_id	Padding token and token id

Table 2: Special tokens and token id attributes[23]

Special tokens can be added at runtime by calling add_special_tokens with a dictionary containing the key additional_special_tokens with the list of new tokens[23].

The main functionality of a tokenizer is to convert text to token ids. This is done by calling the tokenizer directly, as it implements the special __call__ function[23]:

```python
__call__(
    text: Union[str, List[str]],
    add_special_tokens: bool = True,
    return_tensors: str = None,
    // ... omitted parameters
) -> BatchEncoding
```

Listing 4: Tokenizing text

The function in Listing 4 either tokenizes a single string, or a batch of strings. The output BatchEncoding has the field input_ids, which contains the sequence of token ids, or batch of token ids.

The output format of input_ids is controlled by the parameter return_tensors. By setting it to pt, the input_ids will be a PyTorch tensor.

By default, the tokenizer adds special tokens to the input text, such as the BOS and EOS tokens. This behavior can be controlled by the add_special_tokens parameter.

In contrast to tokenization, the PretrainedTokenizer class provides two different methods for detokenization. Both functions are similar in signature with the single difference that one method takes a single sequence of token ids, and the other method a batch of sequences[23].

The method decode decodes a single sequence and has the following signature:

```python
decode(
    token_ids: Union[torch.Tensor, List[int]],
    skip_special_tokens: bool = False,
    // ... omitted parameters
) -> str
```

Listing 5: Detokenizing a single sequence[23]

The method batch_decode has a similar signature, but decodes a whole batch of sequences. The parameter skip_special_tokens can be enabled to remove special tokens, which might be unwanted when e.g. displaying the returned text[23].

If single tokens are already present, these can be convert to token ids by calling the method convert_tokens_to_ids. The converse direction is possible by calling the method convert_ids_to_tokens. The method convert_ids_to_tokens also takes the parameter skip_special_tokens[23].

Config

Models show dynamic behavior, as they can be configured at runtime. This dynamic configuration enables the use of different generation strategies, by setting different model configuration values. Model configurations are classes inheriting from PretrainedConfig. The PretrainedConfig includes configuration values for the model architecture and for the mentioned dynamic behavior[24]. Instantiation a PretrainedConfig is possible through the abstraction AutoClasses, by using AutoConfig[22].

Often single configuration values can be passed into the model as single keyword arguments, without the need for a dedicated configuration object[25]. Different configuration values will be introduced as function arguments in the following sections.

Text Generation

After obtaining a model and a tokenizer, the text generation process can be performed by using one of the following three abstraction. The to be introduced abstraction build upon the previous abstraction.

The lowest level abstraction is the **forward** method of the model, which takes previous token ids and returns the logits for the next token id. The arguments to the function only control the provided output format and cache behavior[21].

The next abstraction is the **generate** method, which generates a whole sequence of tokens, until a stopping criteria is met, such as the EOS token. The functions is provided by the **GenerationMixin** class, which CLM inherit from using multiple-inheritance. The **GenerationMixin** implements the different text generation strategies, which can be controlled through the provided parameters[25]. The **generate** function has the following signature:

```python
generate(
    inputs: torch.Tensor,
    generation_config: GenerationConfig = None,
    logits_processor: LogitsProcessorList = None,
    stopping_criteria: StoppingCriteriaList = None,
    **kwargs
    // ... omitted parameters
) -> Union[ModelOutput, torch.LongTensor]
```

Listing 6: Generating a sequence of tokens[25]

Next to the provided input tensor **inputs**, the **generation_config** parameter contains configuration values to control the generation. Extra matching keyword arguments override keys in the generation config. Table 3 contains important configuration parameters:

Category	Parameter	Type	Description
Stopping Criteria	max_length	int	Maximum length of the generated sequence
	max_new_tokens	int	Maximum number of new tokens to generate. Overrides **max_length**
	max_time	float	Maximum time in seconds to generate
Generation Strategy	do_sample	bool	Whether to sample from the logits
	num_beams	int	Number of beams for beam search
Logits manipulation	temperature	float	Temperature for sampling
	top_k	int	Number of tokens to sample from
	top_p	float	Cumulative probability for sampling from
	repetition_penalty	float	Penalizes repeated tokens

Table 3: Generation configuration parameters[25]

The first parameters in Table 3 configure the first stopping criteria. The generation can be stopped when the sequence reaches a certain length, or when a certain time is reached.

The following diagram displays the different used generation strategies, depending on the provided parameters:

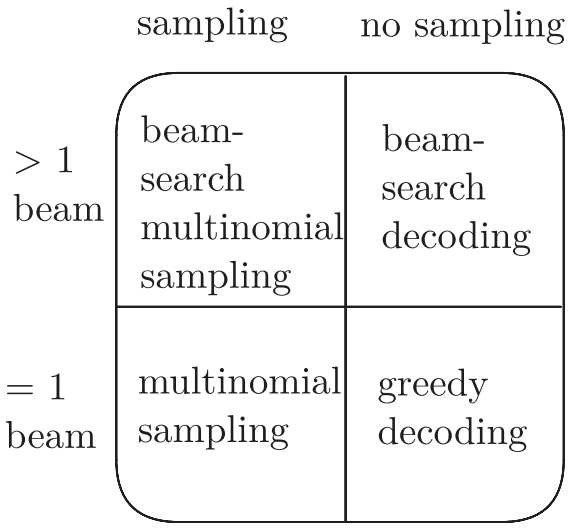


Figure 3: Generation strategies

The final parameters manipulate the logits before sampling. They enable Top-k, Top-p and Temperature sampling. The last parameter hinders models from endlessly repeating the same token, until a stopping criterium is reached.

Controllable Generation

The generation method provides parameters to overview and control the generation process. These parameters enable strategies as discussed in Section 2.3.

The first control mechanism are the stopping criteria. The stopping criteria define custom conditions that terminate the generation, when one of the conditions is met.

Transformers defines a few vital stopping criteria, like the maximum time or the maximum number of tokens to generate. These stopping criteria are enabled by passing certain keyword arguments into the generate method, as introduced in the section before[26], [25].

Custom stopping criteria are defined by subclassing the StoppingCriteria class and adding an instance of the new class to the stopping_criteria list in the generate method[26].

A subclass of StoppingCriteria only needs to implement the __call__ method:

```python
__call__(
    input_ids: torch.LongTensor,
    scores: torch.FloatTensor,
    **kwargs
) -> bool
```

Listing 7: Stopping criteria signature[26]

The method in Listing 7 returns whether the generation should be stopped, based on the previous token ids input_ids and the current scores. The method must support batching, as input_ids has the shape (batch_size, sequence_length) and scores the shape (batch_size, vocab_size)[26].

The second control mechanism are the logits processors. Logits processors manipulate the current logits before a decoding strategy chooses the next token id on the then manipulated logits.

Custom logits processors are defined by subclassing the LogitsProcessor class and adding an instance of the new class to the logits_processor list in the generate method. A subclass of LogitsProcessor only needs to implement the __call__ method:

```python
__call__(
    input_ids: torch.LongTensor,
    scores: torch.FloatTensor,
    **kwargs
) -> torch.FloatTensor
```

Listing 8: Logits processor signature[26]

The arguments in Listing 8 are identical to the arguments in Listing 7. The method must return the manipulated logits, which have the same shape as the input scores.

2.6 Language Server Protocol

Many modern editors and IDEs provide features such as:

- Autocompletion
- Signature help
- Go to the definition of a symbol
- Go to the uses of a symbol
- Provide diagnostics data on the current file
- Rename symbols

This list is not exhaustive, and the features between different editors may vary. These features are vital for any programmer, as the effort to create and maintain code is significantly reduced. Navigating large code bases and finding documentation on used code is highly beneficial.

These many features take a significant effort to implement for a **single** programming language and editor. Providing support for several programming languages massively increases the implementation effort. In addition, the created features must be maintained next to the editor itself. As more programming languages are designed and are adopted, the need for tooling for these programming languages arises. Editor maintainers must then integrate these features for the new programming language.

Providing support for an ever increasing amount of programming languages is hard, but supporting these features for an ever increasing amount of programming languages in every editor is impossible. Many diverse editors exist, such as Emacs[27], [28], VSCode[29], [30], Neovim (modern fork of Vim)[31], [32], Zed (the successor of Atom)[33], [34], Jetbrains IDEs[35], [36], Helix[37], [38] and many more.

Thus, a solution is needed to extract thes **common** features from the editors and provide them centrally, such that every editor need only implement an interface to integrate the common features. The Language Server Protocol is an effort to standardize a common feature set for programming language tooling for use in IDEs. The protocol was designed my Microsoft for use in its Visual Studio Code editor, but is now an open standard. This thesis will focus on version 3.17 of the protocol. The protocol includes many different features, of which the following sections introduce a few important. The full specification may be accessed here°[39].

Design

The protocol is based on a bidirectional client-server architecture. The client is a tool such as the IDE. The server, henceforth called the language server, is programming language specific tooling that provides a subset of the protocol. Both the client and server must not implement the full protocol. The protocol is capability based. Both parties define and communicate their capabilities, then find common capabilities and restrict their use to them. Each client can connect to multiple language servers, and each language server can serve multiple clients. As such a client could use a language server for the Rust

programming language while the user is coding on a Rust project. As soon as the user switches projects to a web project, the editor could then connect to several language servers to provide support for HTML, CSS and JavaScript at once. In that scenario several simultaneous language servers are necessary, as CSS and JavaScript can be embedded into HTML. The communication between the client and server must be bidirectional, as the server must be able to notify the client about events occuring, such as new diagnostic information.

JSON-RPC

The base protocol relies on JavaScript Object Notation-Remote Procedure Call (JSON-RPC). The following section will give a short introduction on the protocol. The content part is specified using JSON-RPC. JSON-RPC is a light-weight and stateless protocol to encode Remote Procedure Calls (RPCs). The specification defines data structures and processing rules. As such, the protocol is transport-agnostic and can be used with any type of message passing framework. The data structures are encoded using the JSON data format.

The protocol is versioned. Currently the versions 1.0 and 2.0 are specified. The LSP uses version 2.0 of JSON-RPC. Henceforth, the sections will focus on version 2.0 of the protocol.

JSON-RPC defines three types of messages:

- Requests
- Responses
- Notifications

, though notifications could be considered a special case of requests. The following section will provide details on each type.

Messages will be defined using TypeScript interfaces, in accordance with the LSP specification. A general message has the following interface:

```typescript
interface Message {
    jsonrpc: string;
}
```

Listing 9: General message interface

To define the other message types, additional TypeScript types are introduced:

- integer: A signed 32-bit integer aliased to the TypeScript type number.
- uinteger: An unsigned 32-bit integer aliased to the TypeScript type number.
- LSPArray:

Request objects have the following JSON format, as defined by the following Typescript interface:

```typescript
interface RequestMessage extends Message {
    method: string;
    params?: any;
    id: string | number | null;
}
```

Listing 10: Request object format

The jsonrpc field contains the version used and must be set to 2.0. The method field contains the name of the remote procedure to be called. Any parameters needed to call the remote function should be passed in the optional field params. At last, the id field identifies the message and thus allows correlation between requests and responses. Using

fractional numbers is discouraged, as the floating point accuracy issue contradicts the identification of the id.

Each request must be answered by a response. The response object has the following format:

```typescript
interface ResponseMessage extends Message {
    result?: any;
    error?: ResponseError;
    id: string | number | null;
}

interface ResponseError {
    code: number;
    message: string;
    data?: any;
}
```

Listing 11: Response object format

Similar to the request object, the jsonrpc field must be set to 2.0. Either the result or error field will be set, depending on the success of the request. To correlate requests and responses, the response will contain the id of the request in the field id.

Error responses contain the error code, a short descriptive message and optional additional data. The specification defines the following error codes and messages:

Code	Message	Meaning
−32700	Parse error	The server received invalid JSON
−32600	Invalid Request	The request is not a valid request object
−32601	Method not found	The method does not exist or is not available
−32602	Invalid params	Invalid method parameters
−32603	Internal error	An internal error occurred while processing the request
−32000	Server error	Reserved for implementation-defined server errors

Table 4: Error codes and messages

In addition, the protocol allows custom error codes.

The final type of message is the notification. Notifications are a special type of request, as they do not contain an id. Notifications do not require a response from the server:

```typescript
interface NotificationMessage extends Message {
    method: string;
    params?: any;
}
```

Listing 12: Notification object format

Base protocol

The base protocol defines the general format, in which messages are sent between the client and server. Each message contains a headers and a content part, separated by \r\n.

The headers part contains at least one header field, which are key-value pairs separated by a colon. Each header field is terminated by a \r\n. The following headers are currently supported:

- Content-Length: A mandatory header that specifies the length of the content part in bytes.

- **Content-Type**: The Multipurpose Internet Mail Extensions (MIME) type of the content part. The default is **application/vscode-jsonrpc; charset=utf-8** and no other charset is currently supported.

The content part is a JSON object, as defined by the JSON-RPC specification. The LSP specification differs from the JSON-RPC in the use of certain types. It restricts the numeric type used for **id** and **code** from **number** to **integer**, to disallow floating point identifiers.

Basic JSON structures

Many RPCs share common structures needed to communicate about language features. The following section introduces a few necessary ones.

The first important structure is the Uniform Resource Identifier (URI), as defined by rfc3986[40]. URIs are passed as strings and are used to uniquely identify files the language server and client work with.

To identify a position inside such a file, the protocol defines the **Position** structure:

```typescript
interface Position {
    line: uinteger;
    character: uinteger;
}
```

Listing 13: Position structure

Line and character define the position in the file starting at **0**.

The last structure is **TextDocumentItem**, which represents an editable text document:

```typescript
interface TextDocumentItem {
    uri: DocumentUri;
    languageId: string;
    version: integer;
    text: string;
}
```

Listing 14: TextDocumentItem structure

In addition to the unique identifier of the document, the **uri**, the structure contains the content of the document in the field **text**. The **languageId** field defines the language of the document. This is needed especially when the language server processes documents of different languages. Passing the **languageId** prevents re-interpreting the file extension. The possible language ids are defined in the specification and can be seen in Table 8 in full.

Lifecycle Messages

After defining the basic structures, this section introduces the first messages to control the lifecycle of the language server.

The first message the client sends the server, is the **initialize** message. No other communication will occur, until the server acknowledges the initialization with the response. The connection between client and server may only be initialized once. The **initialize** message has the following key fields:

```typescript
interface InitializeParams {
  rootUri?: DocumentUri;
  capabilities: ClientCapabilities;
  // ... more fields omitted
}
```

Listing 15: Initialize message parameters

The rootUri is an optional field that specifies the root path of the workspace, the client is working on. Setting the workspace e.g. enables finding virtual environments needed to scan the local code base. The next field capabilities is a complex structure of flags, which specify all capabilities the client has.

The server responds with the initialize result, which contains the server capabilities.

Afer receiving the response, the client must then acknowledge the response, by sending the initialized message. The server then has the option to e.g. register capabilities dynamically or run further initializations. Afterwards, the initialization process is finished.

After using the language server, the client can stop the server in a two step process. First, the client sends a shutdown message. The server then stops itself, but preserves its messaging capabilities to send a proper response. In the next step, the client sends the exit message, after which the server exits its process.

Document Synchronization

After introducing messages to establish and control the connection between server and client, this section focused on the next step, the document synchronization. The goal of these messages is to synchronize the server to the users actions. The server should provide capabilities for a file, while the user works on it.

The first message is the textDocument/didOpen notification. Sending this notification informs the server that the client opened the document. The message has the following structure:

```typescript
interface DidOpenTextDocumentParams {
  textDocument: TextDocumentItem;
}
```

Listing 16: DidOpenTextDocument message parameters

As soon as the file is closed, the client sends the textDocument/didClose notification. The message has the same structure as in Listing 16.

Other messages for synchronizing documents and notebooks exist and are omitted.

Language Features

This section displays messages to query language features. The LSP specification defines many different features, of which this section highlights the few relevant ones:

- Auto Completion
- Signature help
- Provide diagnostics data on the current file

The auto completion feature provides a list of completion items for a given position in a document. The completion items can be selected to provide more detailed information. On applying the selected completion item, the completion text is inserted into the document at the given position. The feature is split into the two messages textDocument/completion and completionItem/resolve.The first message queries the completion items for the given position in a text document. Each completion item only contains basic information. If further details are required, the second message completionItem/resolve asks the server to extend a single completion item with in depth details.

A completion item has the following structure:

```typescript
interface CompletionItem {
    label: string;
    insertText?: string;
    labelDetails?: CompletionItemLabelDetails;
    detail?: string;
    documentation?: string | MarkupContent;
    kind?: CompletionItemKind;
    tags?: CompletionItemTag[];
    deprecated?: boolean;
    preselect?: boolean;
    sortText?: string;
    filterText?: string;
    data?: any;
    // ... more fields omitted
}
```

Listing 17: CompletionItem structure

The first key property is **label**. The **label** is the main text to be inserted into the document and provides the full symbol. In case the document already contains a prefix of the **label** in the given location, the inserted text must not contain the prefix. In these cases the **label** property remains untouched, but **insertText** is set to the shortened text. **insertText** overrides the text to be inserted, but remains optional.

The **labelDetails** property contains additional information, such as the locator of the symbol to be inserted. The structure of the property can be found in Listing 18.

For accessibility, the completion item includes several human readable properties:

- **detail** provides a short description of the completion item.
- **documentation** provides a detailed description of the completion item. The description can be plain text or Markdown. The structure of the property can be found in Listing 20.

The next field **kind** specifies the type of the completion item. The type refers to the symbol kind, such as a "function", "field" or "string". The full list is defined in Listing 30.

Annotations can be found in the field **tags**. The only supported tag is **Deprecated**, as defined in Listing 19. It replaces the now deprecated field **deprecated**.

After receiving the completion items, the client can display the items in a list. The field **preselect** can mark a **single** completion item to be preselected in the list. Additionally, the client may filter and sort the items. In these cases it should use the fields **filterText** and **sortText** for theses tasks, instead of **label**.

Additional server specific data is stored in the field **data**. This information is needed by the server to resolve the completion item and should not be changed.

```typescript
interface CompletionItemLabelDetails {
    detail?: string;
    description?: string;
}
```

Listing 18: CompletionItemLabelDetails structure

```typescript
namespace CompletionItemTag {
    const Deprecated = 1;
}
type CompletionItemTag = 1;
```

Listing 19: CompletionItemTag enum

```typescript
interface MarkupContent {
    kind: MarkupKind;
    value: string;
}
```

Listing 20: MarkupContent structure

```typescript
namespace MarkupKind {
    const PlainText: 'plaintext' = 'plaintext',
    const Markdown: 'markdown' = 'markdown'
}
type MarkupKind = 'plaintext' | 'markdown';
```

Listing 21: MarkupKind enum

The Listing 20 structure includes markup content in the field **value**. The type of markup is defined in the field **kind**. Currently the two kinds **plain text** and **Markdown** are supported, as seen in Listing 21.

To request completions, the client then sends the **textDocument/completion** message. The request the following structure:

```typescript
interface CompletionParams {
    textDocument: TextDocumentIdentifier;
    position: Position;
    context?: CompletionContext;
    // ... more fields omitted
}
```

Listing 22: Completion message parameters

The request contains a **position** in the document **textDocument**. The optional **context** has further details on how the completion was triggered (**triggerKind**) and with which character it was triggered (**triggerCharacter**), as seen in Listing 23. Several trigger kinds are currently supported, such as invoked, triggered by a character, or re-triggered to expand an incomplete completion list (Listing 24).

```typescript
interface CompletionContext {
    triggerKind: CompletionTriggerKind;
    triggerCharacter?: string;
}
```

Listing 23: CompletionContext structure

```typescript
namespace CompletionTriggerKind {
    const Invoked: 1 = 1;
    const TriggerCharacter: 2 = 2;
    const TriggerForIncompleteCompletions: 3 = 3;
}
type CompletionTriggerKind = 1 | 2 | 3;
```

Listing 24: CompletionTriggerKind enum

The server then responds either with no result, directly with a list of completion items, or a completion list, as seen in Listing 25.

```typescript
interface CompletionList {
    isIncomplete: boolean;
    items: CompletionItem[];
    // ... more fields omitted
}
```

Listing 25: CompletionList structure

In addition to containing the completion items, the structure also may indicate that the list is incomplete and will be recomputed through the field **isIncomplete**.

Resolving a single completion item is done by sending the **completionItem/resolve** message. The message is the completion item to be resolved. The server then responds with the resolved completion item. The resolved completion item is a superset of the provided completion item.

The second feature is the signature help. The signature help provides information about the signature of the symbol at the specified position. A common use case is to display function and class parameters.

Sending the **textDocument/signatureHelp** request to the server returns the signature help. The request has a similar structure to Listing 22, as seen in Listing 26:

```typescript
interface SignatureHelpParams {
    textDocument: TextDocumentIdentifier;
    position: Position;
    context?: SignatureHelpContext;
    // ... more fields omitted
}
```

Listing 26: SignatureHelp message parameters

The **context** field contains additional information about the context in which the signature help was triggered.

The server responds with the signature help in the structure of Listing 27:

```typescript
interface SignatureHelp {
    signatures: SignatureInformation[];
    activeSignature?: uinteger;
    activeParameter?: uinteger;
}
```

Listing 27: SignatureHelp structure

The symbol could have several signatures, of which each is represented by a **SignatureInformation** structure. Each single **SignatureInformation** has a **label**, to be shown in the UI, as well as the documentation regarding the whole item. Single parameters can be retrieved through the **parameters** field. When typing a function call, the currently typed parameter becomes the active parameter in the **SignatureInformation** (**activeParameter**). The parameters each come with a **label** and **documentation** field. The **label** either is a string, or a range referring to the label of the outer signature.

The signature help structure is defined in Listing 28, while the single parameter information is provided in Listing 29.

```typescript
interface SignatureInformation {
    label: string;
    documentation?: string | MarkupContent;
    parameters?: ParameterInformation[];
    activeParameter?: uinteger;
}
```

Listing 28: SignatureInformation structure

```typescript
interface ParameterInformation {
    label: string | [uinteger, uinteger];
    documentation?: string | MarkupContent;
}
```

Listing 29: ParameterInformation structure

2.7 Related Work

Several prior works introduce repository level context or global context into neural code completion. The work Monitor-Guided Decoding of Code LMs with Static Analysis of Repository Context [3] by Agrawal et al. is the most advanced framework and uses static analysis to supplement code completion. Other works, such as RepoCoder [41] or DocPrompting [42], use Retrieval Augmented Generation (RAG) techniques to retrieve supplemental context.

Monitor-Guided Decoding of Code LMs with Static Analysis of Repository Context

The work by Agrawal et al. introduces Monitor-Guided Decoding (MGD), a technique to guide the decoding by observing the generation using a monitor, combined with static analysis.

During the generation process, the monitor has access to the current token sequence and the logits of the prediction. The monitor then retrieves information from the static-analysis, based on the current token sequence. Depending on internal rules, the monitor decides if the logits should be guided and to what extent[3].

The work uses the **LSP** to retrieve information about the global context. It suggests rules to guide the generation of the dereference operator ".", names of instantiated classes, the number of arguments in a function call and enumeration options in a switch statement. They show that smaller models can be augmented to outperform larger models through the use of the monitor. Additionally, compilation rates are consistently improved[3].

RepoCoder

The work by Zhang et al. introduces RepoCoder, a framework to improve neural code completion on a repository level through iterative retrieval and generation of code.

RepoCoder is based on a conventional RAG, which uses a similarity-based retriever to locate relevant code snippets in the repository and enhance the code completion using the relevant code. The key insight is to enhance the similarity-based retriever by retrieving code similar to the generated code and regenerating the code with the new retrieved code snippets iteratively[41].

DocPrompting

The work by Zhou et al. introduces DocPrompting, a framework to improve neural code completion by using documentation as context.

In contrast to Zhang et al., the work uses documentation as context to improve code completion and does not work iteratively. The documentation is sourced from globally available documentation, such as DevDocs[43].

3 Design

After introducing the necessary background in Section 2, this section will first derive the requirements from the motivation and goals in Section 3.1.

By analyzing the shortcomings of existing solutions in Section 3.2, the thesis will propose a new design to integrate high quality context into neural code completion. Context retrieval is enhanced with hand crafted filtering rules, to reduce noise and improve completion accuracy.

Lastly, a dataset design is proposed to evaluate the effectiveness of the new design.

3.1 Requirements

The general requirements of this thesis are derived from the motivation and goals, namely the design of a Copilot like solution which has **full** knowledge of the local development environment.

The general requirements in descending order of importance are seen in Table 5.

Identifier	Keyword
R1	Dependency Accuracy
R2	General Correctness
R3	Robustness
R4	Generalizability
R5	Performance

Table 5: General requirements of the thesis

Dependency Accuracy

Neural code completion works under the assumption that the CLM has knowledge of the used code dependencies. This requires either vast amounts of training data on code written using a similar version of the dependencies or the local code to already use the dependency correctly. The first option has the problem, that the training data is not annotated with the correct dependency version, thus CLMs cannot differentiate between different versions of the same dependency. The approach works well for widely used dependencies, which have breaking changes rarely and do not have many different used versions. In contrast, the approach does not work for dependencies which are either too new, or too unused to be known to the CLM. Another problematic type of dependency is the often changing dependency, as the model cannot process breaking changes in the dependency.

The second option, in-context learning, allows the CLM to dynamically learn a small subset of the dependency. The benefit of such a dynamic approach is that the CLM has accurate knowledge of dependency knowledge previously unknown. The huge flaw is that the CLM can only copy the developer, thus only achieving quality on par, but not above the developer. Additionally, the developer has to write the initial code, in which case the developer either misuses the dependency or already acquired knowledge of the dependency, in which case the CLM cannot support him in that regard.

1. The proposed approach shall provide neural code completions with a high accuracy, by always considering the **exact local development environment** and the full featureset of each dependency.
2. It shall differentiate between nuances of different versions of the same dependency.

3. To enable diverse code completions, the approach shall be able to handle a wide range of dependencies, even if they have never been used in other public projects before.

The second dependency accuracy requirement has the implication that the approach should follow (some) clean code practices. Dependencies use a "Deprecation" mechanism to gradually change features, without breaking dependant code directly by providing a time window, in which the code can be changed, before the old feature is removed or replaced. By differentiating between nuances in different versions, the approach can suggest code which is future-proof and does not rely on deprecated features. The following different scenarios between dependency versions can be distinguished:
1. The dependency has a new feature, which is not available in the old version.
2. The dependency has a feature, which is deprecated in the new version and will either be removed or replaced.
3. The dependency has a feature, which is not available in the new version.

It should be notes, that local code bases are a special case of a dependency and should not be treated differently, as many different works do. Locally available code is also imported and used alongside remote dependencies. As such, the **scope** of this thesis includes prediction of all possible code.

General Correctness

The proposed approach builds upon other work to improve especially dependency handling in neural code completion. Still, the basic assumptions on code correctness should still stand. In particular, the approach should suggest syntactically correct code. Additionally, the approach should suggest code that is functionally correct and solves the given task.

Both correctness requirements are the essentials, which are expected from any code completion tool, upon which the thesis improves. The focus of this thesis lies on improving methodical correctness through Dependency Accuracy and functional correctness and General Correctness are secondary.

Robustness

Developers may work on code in a variety of different environments and using different types of dependencies. The approach should be robust under any circumstances and still provide reasonably good completions.

Specifically, the approach should be able to handle the following scenarios:
1. The size of the project should be irrelevant. The approach should scale to any project size.
2. The approach should be able to handle any type of dependency, irrelevant of the size or complexity of the dependency. It shall not matter if the dependency has no documentation or has pages full of documentation.

Generalizability

The programming language ecosystem is vast and diverse. Even though only a few programming languages are widely used[44], the approach should be able to handle most programming languages. By supporting lesser used programming languages, this approach can be used to help developers in a wider range of projects, such as hobby projects. It could be even used to maintain legacy code.

Performance

Using editor tooling feels fast, as responses in interactions with toolings only take a few milliseconds, up to a second. Integrating CLM for code completion increases the latency by a significant margin, as inference takes several seconds up to a minute.

While the goal of this thesis is not to reduce the latency in neural code generation, the approach should not impose a significant overhead on the default code completion.

3.2 Analysis

After introducing the five requirements in the previous section, this section will analyze the shortcomings of existing solutions in regards to the requirements. In particular, the analysis will focus on the work by Agrawal et al.[3], as it is the most promising work. While this thesis has evolved without the knowledge of the work by Agrawal et al. and is not based on it, the general design choices are similar enough to show initial observations and design choices.

Monitor-Guided Decoding

In short, the work by Agrawal et al. fulfills all requirements except the Dependency Accuracy, but is limited in scope when considering dependencies.

By guiding the neural code completion selectively through the monitor, the approach improves upon the default code completion and fulfills the requirement of General Correctness. Meanwhile, the Dependency Accuracy is only partially fulfilled. The work takes an important step in using LSP, as it enables retrieving context globally of the exact version of used dependencies. As the work only guides the dereference operation and function arguments count, the approach does not consider changes between dependency versions, such as deprecated items. It cannot guide the model away from now deprecated features.

Additionally, other code items, such as function signature parameters are not considered in the work.

As the work depends on the LSP, it is robust under the assumption, that the running language server is robust. By abstracting language dependant features, the work is generalizable to other programming languages, though the implementation of the work relies on Java specific functionality. Through the use of MGD, the only performance overhead of this method is the additional processing for each token. The bulk of the processing is shifted towards the language server, which performs the static analysis. As the language servers are optimized towards these analysis, the performance impact is $O(n)$, where n is the token count[3].

RepoCoder

The work by Zhang et al. fails in most requirements, only fulfilling basic requirements. It improves the default neural code completion through the use of repository level code snippets, thus fulfilling the General Correctness requirement. But in only considering repository level code, it fails to work on remote dependencies and cannot capture the nuances between different versions of dependencies. Thus, it also fails the Robustness, especially as embedding of the local repository must be performed upfront. This upfront embedding of local code scales with the repository size and taking a considerable time for larger projects. By reusing local code, the approach is generalizable to other programming languages, as the retrieved context is of the same language. Under the assumption taht the embedding is performed upfront, the performance decreases by the iteration count i.

DocPrompting

The work by Zhou et al. fails similarly to RepoCoder, as it uses a similar approach, but tuned towards globally available documentation. As no iterations are used, the performance impact is a single similarity seach using embeddings.

Summary

While DocPrompting and RepoCoder show that using documentation and available code can improve neural code completion, they fail to improve completions for very specific installed dependenecies. The work by Agrawal et al. fulfills the most requirements, through the clever use of LSP. LSP is a protocol abstracting programming language features, thus enabling generalizable approaches using global context of the exact installed dependencies.

A possible approach would use available information, such as the CompletionItemTag in a CompletionItem, to check if an item was deprecated and should not be used anymore. Furthermore, additional language features, such as signature completions, could be used to guide the neural code completion of parameters in function calls.

A full MGD is not possible, due to deprecated completion items. A typical MGD approach would keep the logits of completion item to guide the dereferece operation, as seen in Agrawal et al., but mask other tokens.

When combined with deprecated completion items, tokens of deprecated completion items would need to be masked, **while** tokens of non deprecated completion items would not be masked. The problem arises when deprecated and non deprecated completion items share the same prefix when represented as a token sequence. Now the same token would need masking and retaining.

The thesis will thus propose a new alternative design, which enables the use of both types of information sources, by reranking logits of the CLM locally in the monitor and globally by altering the prompt with injected information. The design is threefold and includes a novel meta-strategy to guide the model through prompt-engineering, with dynamically obtained information from static analysis. The second part is the approach based on the first novel meta-strategy. Lastly, the design of a new dataset is introduced, to evaluate the performance of the proposed approach.

3.3 Interrupt-based meta-strategy

As sketched in Section 1.3, the Interrupt-based meta-strategy (IBMS) is a novel approach to integrate context into model generation. It is heavily focused on integration multiple information sources independently. The context shall be integrated seamlessly, without affecting the generation flow of the initial generation, except providing the additional information. The strategy shall serve as a basis for globally guiding the model generation, by dynamically injecting relevant context into the prompt.

Requirements

The interrupt-based meta-strategy is just a single part in the overall approach. Thus, the requirements are separate from the general requirements.

The exact requirements are listed in Table 6.

Identifier	Keyword
$\mathbf{R_{IBMS}1}$	Multiple source support
$\mathbf{R_{IBMS}2}$	Flexible context location
$\mathbf{R_{IBMS}3}$	Minimal overhead

Table 6: Special requirements of the IBMS

Multiple source support. The goal of the strategy is to integrate context of multiple sources. As such, the strategy must be able to differentiate between different sources and process these independently.

Flexible context location. In addition to integrating multiple sources, the integrated

context should be able to be placed anywhere in the prompt. It could be integrated into the system prompt, the user prompt or even the model output at "runtime".

Minimal overhead. The last requirement is for the strategy to have minimal overhead over the default model generation. Integrating context should not affect the generation flow of the model in any way, not even performance wise.

Heritage

While previous sections introduced the need for this strategy, this section will discuss the observations leading to the design of the strategy.

While the strategy was developed independently of Agrawal et al.[3], the design uses the same observation made for the Monitor component in their work. The Monitor not only allows controlling model logits at runtime, but it allows **observing the generation in realtime**. This allows a fine-grained control over the model generation, which is not possible with general prompt engineering or meta-strategies. Coupled with any form of static analysis, the monitor can find additional relevant context. This is similar to the iterative approach of RepoCoder. RepoCoder uses each iteration to retrieve relevant code based on the current whole generation, while this observation leads to an approach which finds context during a single generation.

The second observation is that the Monitor is only able to influence the generation on a token by token basis, but cannot guide the whole flow of the generation. Meta-strategies and prompt engineering can guide the whole flow of the generation, but cannot retrieve relevant information efficiently.

The third key observation bridges the gap between meta-strategies and fine-grained control from the Monitor. Additional tokens can be added to the vocabulary of a tokenizer for inference and the model embeddings can be resized to fit the new token. The model will not be able to meaningfully predict the added token, as it was not trained on the ad-hoc token, but strategies can use the added token to elicit behavior. For example, the token can be used as an additional EOS token, stopping the generation process, but with the added semantic information of the special token.

Combining the aforementioned observations, the IBMS combines the fine granularity needed to monitor the generation in realtime, with the prompt editing capabilities of multi-step meta-strategies, through the use of an added special token.

Components

An overview over the strategy is given in Figure 4. All components in blue are additions to the normal model generation.

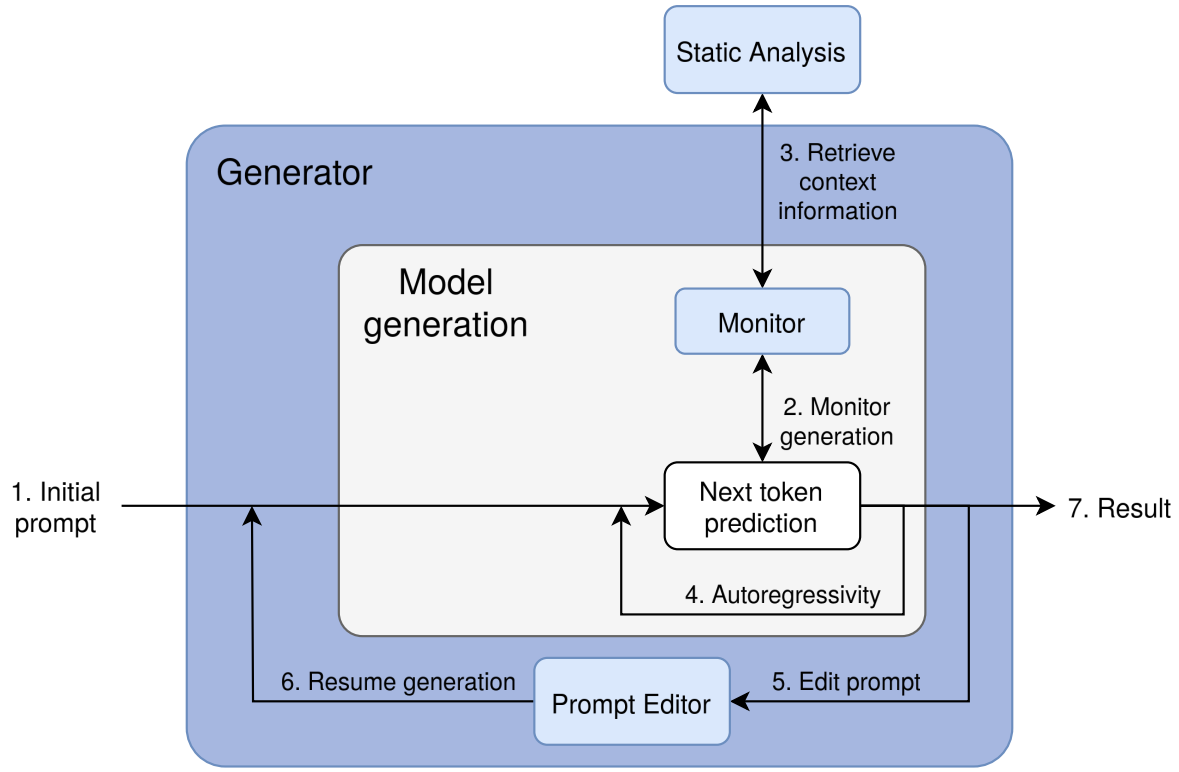


Figure 4: Components of the IBMS

In summary, the components **Generator**, **PromptEditor**, **Monitor** and **Static Analysis** allow context interruption through a custom interrupt special token, prompt edits and resumption through a controlled model generation loop.

The first main component is the **Generator**. It is the main component initializing and orchestrating the different components. To generate a response using the IBMS, first a **Generator** must be initialized. It in turn will initialize the **Monitor**, the **Prompt Editor** and a connection to external **Static Analysis** tooling.

In addition, it will initialize the tokenizer and model, adding a custom special token to the vocabulary of the tokenizer and resize the model embeddings to fit the new special token. The new special token is called the interrupt token and only be used during the approach. No further training with the token is necessary, as the model will not need to predict the tokens, as the **Monitor** component will force the prediction of the interrupt token.

The **Generator** will start the model generation process, providing the initial prompt to the model. After the model finishes the whole generation, it will check the final token. If the final token is **not** an interrupt token, the generation process finished normally and the final generation result can be obtained by removing any remaining injected context through the **PromptEditor**. Else the generation was interrupted with additional context and the **PromptEditor** can edit the prompt with the context. To allow the continuation of the generation, the final interrupt token is removed from the generated token sequence.

Then the next component, the **PromptEditor** is called. It will read the retrieved context and edit the prompt in any way. To edit the prompt, the **PromptEditor** must work on the textual prompt, by detokenizing the prompt, applying any edits and tokenizing the prompt again. To perform edits such as deletions, the **PromptEditor** must be aware of the location and type of inserted context. Thus, it stores a list of injected context, each with their location and type.

To inject context, the editor inserts the textual representation of the context at a location, then saving the location and type of the context. If the location is before any

other injected context, the locations of the following contexts shift forward by the length of the textual representation of the injected context and must be changed accordingly.

Removing context is done similarly, by finding context in the list of all injected contexts, removing it from the stored location and shifting the location of all following contexts back by the length of the removed context.

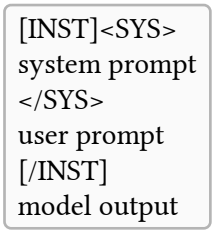


Figure 5: Format of a typical model generation result

Figure 5 shows the format of a (partial) model generation output. The editor is flexible and can edit the prompt in any way, even inserting context into the partial model output.

The edited prompt is then passed into the model generation again, repeating the generation process.

Inside the model generation, the component Monitor watches the generation process, deciding if an interrupt is necessary. It was injected into the default model generation process by the Generator. The Monitor is called for each token prediction, receiving the previous token sequence and the next token prediction and optionally modifying the prediction logits. In its functionality it is similar to the monitor in Agrawal et al.[3], but with the difference that it only controls the prediction of the special interrupt token. If an interrupt is necessary, the Monitor will force the prediction of the interrupt token. In addition, it will store the retrieved context for access by the Generator and PromptEditor after interruption.

To decide if an interrupt is necessary, the Monitor will call an external Static Analysis tool. The Static Analysis component represents any form of external tooling that can analyze the current generation and provide additional context. This could include a web search of named entities, a keyword search in local documents or LSP tooling. If no additional context was found, the model generation proceeds normally. Next to deciding if an interrupt is necessary to add context, the Monitor may also decide to remove old context. The interruption process is similar, with the difference in the stored context information.

To reduce the number of interruptions, the Monitor will only interrupt the generation if the additional context is not yet present in the prompt. To allow interruption of the model generation, the Generator adds a stop condition to the model generation loop. The stop condition checks for the existence of the interrupt token in the generated token sequence. As soon as the Monitor forces an interruption, the stop condition stops the model generation.

Generation Strategy Support

The design mentioned in the previous section uses multiple model generation steps to arrive at a final result. This is similar to a typical multi-step meta-strategy, but with the difference that no major changes are made to the prompt. Each prompt to the model is an edited version of the previous prompt, with the only difference being the injected context.

Moreover, the strategy has a tighter control over the model generation, through the use of the interrupt token **and** support for model generation strategies. The previous section simplified the resumption of the model generation behavior, suggesting that the model generation can be resumed directly after interruption and context injection. This

is the case for simple decoding strategies like greedy decoding, which generate on a single beam.

Complex decoding strategies like beam search require complex handling, as they operate on multiple beams. At the start of a typical generation process using beam search, firstly k hypothesis of the input prompt are initialized. Then the generation process is started, with each hypothesis predicting the next token to form $k * V$ combinations, of which the top-k combinations are kept.

When the stop condition is met, the **singular** top hypothesis is selected as the generation result. The Generator must ensure that the interrupted generation returns the current top-k hypothesis and is resumed with the edited top-k hypothesis.

To support complex decoding strategies, the Generator must use an edited model generation loop, that can store all beams currently in the generation process and which can resume the generation process with existing beams. Additionally, interrupts occur on a single beam, not affecting other beams. Each component storing state must store the state for each beam separately. E.g. the PromptEditor will store the injected context for each beam separately.

Furthermore, the Generator must ensure the correct order of beams, as the order of beams can change during the generation process, e.g. in the top-k selection process in the beam search, as it may keep the top-k beams sorted descending. After interruption, the then current top-k beams originate from $l \leq k$ original beams, where l depends on the spread of selected beams. If each of the k beams is continued at each step, then $l = k$. If the top-k beams all originate from a single beam, then $l = 1$.

3.4 Language Server Protocol aided generation

The LSP aided generation builds upon the interrupt-based meta-strategy. It applies the strategy to neural code completion, injecting context information from the LSP server. In addition to injecting context into the prompt, the logits in the next token prediction are controlled using the context, to improve the robustness of the work.

This section will first talk about the heritage, introduce the components of the design and lastly discuss design decisions.

Heritage

The key thought guiding the work is:

> Developers base their decisions on contextual information provided by the LSP server.
> Why can't the neural code completion do the same?

Developers use a wide variety of tools to guide their programming decisions. Code completion is the most frequently used tool inside IDEs[1]. It displays available symbols, their documentation and type information. Next to code completion, signature help is another important tool, as it displays available parameters of selected functions and their types, thus providing information about the structure of the data. Additionally, IDEs highlight the code with diagnostics information, such as important errors or warnings which may be ignored. The diagnostics information guide the developer to potential problems, such wrong symbols, or code smells, such as dead code.

Components

An overview over the work is given in Figure 6. All components in blue are additions to the normal model generation.

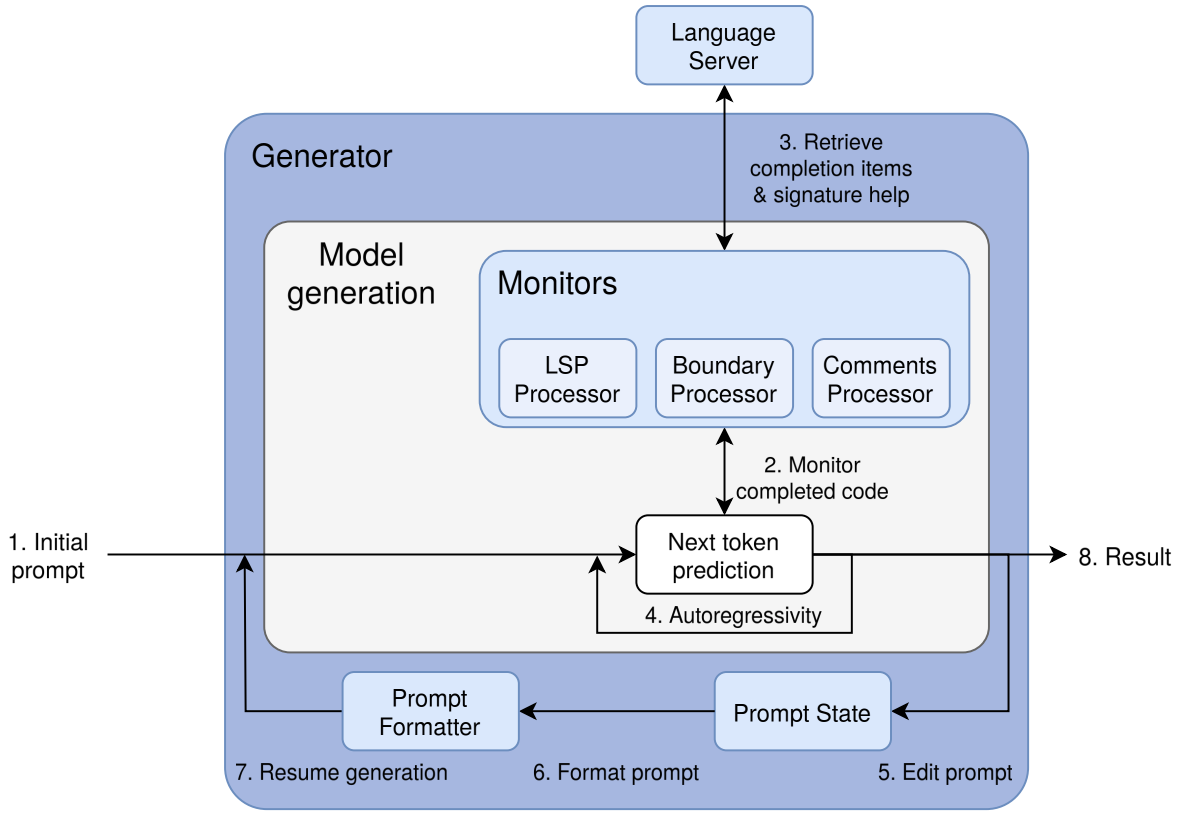


Figure 6: Components of the LSP aided generation

As the work is based on the interrupt-based meta-strategy, the components are similar to the ones in Figure 4. The main difference is the added complexity in the previous **Monitor** component, which now features several distinct subcomponents. Other differences are the specific use of LSP and a more complex prompt processing.

The **Generator** component depicted in Figure 6 is more specific in handling the prompt and injected context. The previous component **Prompt Editor** is now replaced with the two more specific components **Prompt State** and **Prompt Formatter**.

The **Prompt State** component contains a structured representation of the initial prompt and all injected context information. It is a general utility class, which is used in the **Monitors** and in the **Generator** to extract the completed code and to reconstruct the edited prompt. Extraction of the completed code is possible by detokenizing the partial result and removing the initial prompt prefix. As often the detokenization is not the inverse of the tokenization, the **Prompt State** class includes fixes for the detokenization. To reconstruct the edited prompt, the **Prompt State** passes the initial prompt and injected contexts to the **Prompt Formatter**.

The **Prompt Formatter** abstracts different prompt formats used in different models. For Llama based models, it would use the format defined in Figure 5. Different models allow different prompt sections, such as a system and a user prompt. Depending on the availability of such sections, the component decides how to embed the context.

The difference in the **Monitors** is the threefold divide into subcomponents. The first subcomponent **LSP Processor** corresponds to the previous component **Monitor**. It receives context from a **Language Server** (previous **Static Analysis**) and enables the interrupt-based meta-strategy applied to code completion. The task of the **LSP Processor** is twofold. First, it enables the interrupt-based meta-strategy. Secondly, it controls the logits to force the model towards certain behavior, when guiding is not enough.

The purpose of the **Boundary Processor** is to fix boundary issues when using tokenizers which allow multi-character tokens. Usually, LSP information, such as autocompletion and

signature help, is requested after every keystroke in an IDE, or for certain hotkeys. The autocompletion information is especially useful after the dereference operator (. in Python, -> in C++), as then available symbols are displayed. The signature help information is useful after starting to write a function call, as then the available parameters are displayed.

As tokenizers allow tokens with several characters, the vocabulary could include tokens that start with a trigger character, followed by any tail. For autocompletion, this could include tokens such as .model. For signature help, this could be a token such as (). By predicting these tokens, the completed code would "jump" over the critical trigger characters, thus ignoring the autocompletion or signature help information.

The Boundary Processor fixes this issue by identifying all tokens prefixed with a trigger character and replacing these tokens with the trigger character only. It ensures that the logits value of the trigger character is higher than any other token containing the trigger character as a prefix.

Next to the Boundary Processor, the Comments Processor solves an issue regarding the generation of comments. A possible place to inject the context is in the completed code, as a code comment. The problem is that a model could see the code comments after an interrupt and try to predict a code comment preceding the next code block. Thus, code comments could be tainted by the code prediction itself The Comments Processor disallows the generation of code comments, by stopping the prediction of the token starting a code comment by setting the logits value to $-\infty$.

Design Decisions

This section provides details on different design decision for the introduced components.

The first design decision is the selection of sources to suggest fitting code, as introduced in **RQ1**°. The previous sections already highlight why the LSP is fitting. The protocol provides a vast amount of different information. We argue that the core information to use are the completion items and the signature help information.

When generating code, the primary source are the completion items, as they dictate the available symbols. The completion items list the attributes of objects and which attributes are deprecated and should be replaced.

After using the completion items to write callable code, the secondary source is the signature help. Likewise, the signature help lists available parameters of a function call and includes documentation.

Both types of information sources allow code generation using up-to-date information. Other features provided through LSP may improve code generation, but are not vital, such as diagnostics information. In addition, both sources can be used on partial code and provide suggestions on how to extend the code, aiding the natural CLM generation flow. Diagnostic information and other information reference code already written, increasing the difficulty to integrate into the natural generation flow.

The second design decision, **RQ2**°, is about processing the completion items and signature help information. Both sources provide critical information, but also a lot of unrelevant information not needed to decide on the next code piece. Such context pollutes the context window of CLM and worsens the prediction quality. As such, preprocessing is needed to filter such data.

Completion items are initially filtered by their CompletionItemKind[3]. Only items of kind Method, Field, Class, Function, Property or Variable are allowed, to remove kinds such as Snippet, Color and File.

To reduce confusion further, completion items of builtins are removed, as the assumption is made that the model has the knowledge of builtins. If the model does not

[3]The full list is visible in Listing 30

have knowledge about builtins, this filter can be disabled, to e.g improve support of low resource languages. Filtering is done by checking the detail field of the completion item, as it contains the origin of the code item, such as the code file.

Next, completion items from the current code piece are removed. Similarily, it is assumed that the current code is in the context of the CLM. Again, this filter checks the detail field of the completion item.

Then private or meta functions are removed, such as the __getitem__ function in Python. These functions are not meant to be called directly.

To avoid the completion of already completed code, the to be inserted text is checked against the ending of the currently generated code. This also removes items which do not properly extend the current code.

Then the completion items are split into two categories, the non-deprecated and the deprecated completion items. This differentiation allows later steps to guide the model towards non-deprecated items, while avoiding deprecated items.

The model may be biased towards generating old deprecated code. To only focus on the deprecated items, which the model **may** predict, the deprecated items are finally filtered by the next predicted token.

4 Implementation

4.1 Interrupt-based meta-strategy

4.2 Language Server Protocol aided generation

4.3 Dataset design

5 Results

5.1 Collection method

5.2 Possible configurations

5.3 Results

6 Evaluation

6.1 Research Questions

Identifier	Keyword	Description
RQ1	Information source selection	Which sources provide CLMs with the necessary information to suggest fitting code?
RQ2	Context processing	How to filter sourced information to provide the most relevant information to CLMs?
RQ3	Context integration	How to integrate the contextual information into the neural code completion?
RQ4	Robustness	How to ensure the robustness of the approach?
RQ5	Generalizability	How to generalize the approach to work with different programming languages?
RQ6	Performance	How to reduce the performance impact of the approach?

Table 7: Research questions guiding the thesis.

6.2 Evaluation difficulties

6.3 Evaluation

7 Conclusion

7.1 Recap

7.2 Outlook

Acronyms

BOS: Begin of Sequence

BPE: Byte Pair Encoding

CLM: Causal Language Model

Code LLM: Large Language Model for Code

EOS: End of Sequence

IBMS: Interrupt-based meta-strategy

IDE: Integrated Development Environment

JSON: JavaScript Object Notation

JSON-RPC: JavaScript Object Notation-Remote Procedure Call

LLM: Large Language Models

LM: Language Model

LSP: Language Server Protocol

MGD: Monitor-Guided Decoding

MIME: Multipurpose Internet Mail Extensions

PAD: Padding

RAG: Retrieval Augmented Generation

RPC: Remote Procedure Call

URI: Uniform Resource Identifier

Bibliography

[1] S. Amann, S. Proksch, S. Nadi, and M. Mezini, "A Study of Visual Studio Usage in Practice," in *2016 IEEE 23rd International Conference on Software Analysis, Evolution, and Reengineering (SANER)*, Suita: IEEE, Mar. 2016, pp. 124–134. doi: 10.1109/SANER.2016.39°.

[2] A. Ziegler *et al.*, "Productivity Assessment of Neural Code Completion," in *Proceedings of the 6th ACM SIGPLAN International Symposium on Machine Programming*, San Diego CA USA: ACM, Jun. 2022, pp. 21–29. doi: 10.1145/3520312.3534864°.

[3] L. A. Agrawal, A. Kanade, N. Goyal, S. K. Lahiri, and S. K. Rajamani, "Monitor-Guided Decoding of Code LMs with Static Analysis of Repository Context."

[4] "Speech and Language Processing." Accessed: Jun. 04, 2024. [Online]. Available: https://web.stanford.edu/~jurafsky/slp3/°

[5] "ChatGPT." Accessed: Jun. 10, 2024. [Online]. Available: https://chatgpt.com/°

[6] "https://home.csulb.edu/~cwallis/382/readings/482/mccolloch.logical.calculus.ideas.1943.pdf." Accessed: Jun. 10, 2024. [Online]. Available: https://home.csulb.edu/~cwallis/382/readings/482/mccolloch.logical.calculus.ideas.1943.pdf°

[7] L. Weng, "Controllable Neural Text Generation." Accessed: Jun. 12, 2024. [Online]. Available: https://lilianweng.github.io/posts/2021-01-02-controllable-text-generation/°

[8] T. B. Brown *et al.*, "Language Models Are Few-Shot Learners." Accessed: Jun. 12, 2024. [Online]. Available: http://arxiv.org/abs/2005.14165°

[9] J. Wei *et al.*, "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models." Accessed: Oct. 10, 2023. [Online]. Available: http://arxiv.org/abs/2201.11903°

[10] S. Yao *et al.*, "Tree of Thoughts: Deliberate Problem Solving with Large Language Models." Accessed: Oct. 10, 2023. [Online]. Available: http://arxiv.org/abs/2305.10601°

[11] S. Rabanser, O. Shchur, and S. Günnemann, "Introduction to Tensor Decompositions and Their Applications in Machine Learning." Accessed: Jun. 20, 2024. [Online]. Available: http://arxiv.org/abs/1711.10781°

[12] "Tensor Attributes — PyTorch 2.2 Documentation." Accessed: Jul. 01, 2024. [Online]. Available: https://pytorch.org/docs/2.2/tensor_attributes.html°

[13] "Torch.Tensor — PyTorch 2.2 Documentation." Accessed: Jul. 01, 2024. [Online]. Available: https://pytorch.org/docs/2.2/generated/torch.tensor.html#torch.tensor°

[14] "IEEE Standards Association." Accessed: Jun. 20, 2024. [Online]. Available: https://standards.ieee.org/°

[15] "BFloat16: The Secret to High Performance on Cloud TPUs." Accessed: Jun. 20, 2024. [Online]. Available: https://cloud.google.com/blog/products/ai-machine-learning/bfloat16-the-secret-to-high-performance-on-cloud-tpus°

[16] "Torch.Tensor.to — PyTorch 2.3 Documentation." Accessed: Jul. 01, 2024. [Online]. Available: https://pytorch.org/docs/stable/generated/torch.Tensor.to.html°

[17] "Torch.Tensor.Dim — PyTorch 2.2 Documentation." Accessed: Jul. 01, 2024. [Online]. Available: https://pytorch.org/docs/2.2/generated/torch.Tensor.dim.html°

[18] “Torch.Tensor.Shape — PyTorch 2.2 Documentation.” Accessed: Jul. 01, 2024. [Online]. Available: https://pytorch.org/docs/2.2/generated/torch.Tensor.shape.html

[19] “Tensor Indexing API — PyTorch Main Documentation.” Accessed: Jul. 01, 2024. [Online]. Available: https://pytorch.org/cppdocs/notes/tensor_indexing.html

[20] “ Transformers.” Accessed: Jun. 18, 2024. [Online]. Available: https://huggingface.co/docs/transformers/v4.41.3/en/index

[21] “Models.” Accessed: Jul. 01, 2024. [Online]. Available: https://huggingface.co/docs/transformers/v4.41.3/en/main_classes/model

[22] “Auto Classes.” Accessed: Jul. 01, 2024. [Online]. Available: https://huggingface.co/docs/transformers/v4.41.3/en/model_doc/auto

[23] “Tokenizer.” Accessed: Jul. 01, 2024. [Online]. Available: https://huggingface.co/docs/transformers/v4.41.3/en/main_classes/tokenizer

[24] “Configuration.” Accessed: Jul. 01, 2024. [Online]. Available: https://huggingface.co/docs/transformers/v4.41.3/en/main_classes/configuration

[25] “Generation.” Accessed: Jul. 01, 2024. [Online]. Available: https://huggingface.co/docs/transformers/v4.41.3/en/main_classes/text_generation

[26] “Utilities for Generation.” Accessed: Jul. 01, 2024. [Online]. Available: https://huggingface.co/docs/transformers/v4.41.3/en/internal/generation_utils

[27] “GNU Emacs - GNU Project.” Accessed: Jul. 01, 2024. [Online]. Available: https://www.gnu.org/software/emacs/

[28] “LSP Mode - Language Server Protocol Support for Emacs - LSP Mode - LSP Support for Emacs.” Accessed: Jul. 01, 2024. [Online]. Available: https://emacs-lsp.github.io/lsp-mode/

[29] “Visual Studio Code - Code Editing. Redefined.” Accessed: Jul. 01, 2024. [Online]. Available: https://code.visualstudio.com/

[30] “Language Server Extension Guide.” Accessed: Jul. 01, 2024. [Online]. Available: https://code.visualstudio.com/api/language-extensions/language-server-extension-guide

[31] “Home - Neovim.” Accessed: Jul. 01, 2024. [Online]. Available: https://neovim.io/

[32] “Lsp - Neovim Docs.” Accessed: Jul. 01, 2024. [Online]. Available: https://neovim.io/doc/user/lsp.html

[33] “Zed - Code at the Speed of Thought.” Accessed: Jul. 01, 2024. [Online]. Available: https://zed.dev/

[34] “Adding New Languages - Zed.” Accessed: Jul. 01, 2024. [Online]. Available: https://zed.dev/docs/adding-new-languages

[35] “JetBrains IDEs: Enjoy an Exceptional Developer Experience.” Accessed: Jul. 01, 2024. [Online]. Available: https://www.jetbrains.com/ides/

[36] “Language Server Protocol (LSP) | IntelliJ Platform Plugin SDK.” Accessed: Jul. 01, 2024. [Online]. Available: https://plugins.jetbrains.com/docs/intellij/language-server-protocol.html

[37] “Helix-Editor/Helix.” Accessed: Jul. 01, 2024. [Online]. Available: https://github.com/helix-editor/helix

[38] "Language Support." Accessed: Jul. 01, 2024. [Online]. Available: https://docs.helix-editor.com/lang-support.html°

[39] "Specification." Accessed: Jul. 01, 2024. [Online]. Available: https://microsoft.github.io/language-server-protocol/specifications/lsp/3.17/specification/°

[40] T. Berners-Lee, R. T. Fielding, and L. M. Masinter, "Uniform Resource Identifier (URI): Generic Syntax," Jan. 2005. doi: 10.17487/RFC3986°.

[41] F. Zhang *et al.*, "RepoCoder: Repository-Level Code Completion Through Iterative Retrieval and Generation." Accessed: Dec. 05, 2023. [Online]. Available: http://arxiv.org/abs/2303.12570°

[42] S. Zhou, U. Alon, F. F. Xu, Z. Wang, Z. Jiang, and G. Neubig, "DocPrompting: Generating Code by Retrieving the Docs." Accessed: Jun. 23, 2024. [Online]. Available: http://arxiv.org/abs/2207.05987°

[43] "DevDocs." Accessed: Jul. 04, 2024. [Online]. Available: https://devdocs.io/°

[44] "Stack Overflow Developer Survey 2023." Accessed: Jul. 03, 2024. [Online]. Available: https://survey.stackoverflow.co/2023/?utm_source=social-share&utm_medium=social&utm_campaign=dev-survey-2023°

Listings

List of Figures

Figure 1: IEEE 754 half-precision floating point number: float16 11
Figure 2: Brain Floating Point Format: bfloat16 11
Figure 3: Generation strategies 16
Figure 4: Components of the IBMS 31
Figure 5: Format of a typical model generation result 32
Figure 6: Components of the LSP aided generation 34

List of Tables

Table 1: Relevant PyTorch data types 11
Table 2: Special tokens and token id attributes 13
Table 3: Generation configuration parameters 15
Table 4: Error codes and messages 19
Table 5: General requirements of the thesis 26
Table 6: Special requirements of the IBMS 29
Table 7: Research questions guiding the thesis. 39
Table 8: Possible language ids defined in the LSP specification 47

List of Code

Listing 1: Loading a pre-trained model 12
Listing 2: Using AutoClasses to load a model 13
Listing 3: Loading a tokenizer 13
Listing 4: Tokenizing text 14
Listing 5: Detokenizing a single sequence 14
Listing 6: Generating a sequence of tokens 15
Listing 7: Stopping criteria signature 16
Listing 8: Logits processor signature 17
Listing 9: General message interface 18
Listing 10: Request object format 18
Listing 11: Response object format 19
Listing 12: Notification object format 19
Listing 13: Position structure 20
Listing 14: TextDocumentItem structure 20
Listing 15: Initialize message parameters 21
Listing 16: DidOpenTextDocument message parameters 21
Listing 17: CompletionItem structure 22
Listing 18: CompletionItemLabelDetails structure 22
Listing 19: CompletionItemTag enum 23
Listing 20: MarkupContent structure 23
Listing 21: MarkupKind enum 23
Listing 22: Completion message parameters 23
Listing 23: CompletionContext structure 23
Listing 24: CompletionTriggerKind enum 23
Listing 25: CompletionList structure 24
Listing 26: SignatureHelp message parameters 24
Listing 27: SignatureHelp structure 24

Listing 28: SignatureInformation structure 25
Listing 29: ParameterInformation structure 25
Listing 30: CompletionItemKind enum 48

List of Examples

Example 1: Language model predicting the last word of the famous English pangram. 4
Example 2: Natural language encoded into tokens. 5
Example 3: Enforcing "yes" or "no" answers 7
Example 4: Few-shot prompting 8
Example 5: Context injection 9

Appendix

Language	Language ID
ABAP	abap
Windows Bat	bat
BibTeX	bibtex
Clojure	clojure
Coffeescript	coffeescript
C	c
C++	cpp
C#	csharp
CSS	css
Diff	diff
Dart	dart
Dockerfile	dockerfile
Elixir	elixir
Erland	erlang
F#	fsharp
Git	git-commit and git-rebase
Go	go
Groovy	groovy
Handlebars	handlebars
HTML	html
Ini	ini
Java	java
JavaScript	javascript
JavaScript React	javascriptreact
JSON	json
LaTeX	latex
Less	less
Lua	lua
Makefile	makefile
Markdown	markdown
Objective-C	objective-c
Objective-C++	objective-cpp
Perl	perl
Perl 6	perl6
PHP	php
Powershell	powershell
Pug	jade
Python	python
R	r
Razor	razor
Ruby	ruby
Rust	rust
SCSS	scss or sass
Scala	scala
ShaderLab	shaderlab
Shellscript	shellscript
SQL	sql
Swift	swift
TypeScript	typescript
TypeScript React	typescriptreact
TeX	tex
Visual Basic	vb
XML	xml
XSL	xsl
YAML	yaml

Table 8: Possible language ids defined in the LSP specification

```typescript
namespace CompletionItemKind {
    const Text: 1;
    const Method: 2;
    const Function: 3;
    const Constructor: 4;
    const Field: 5;
    const Variable: 6;
    const Class: 7;
    const Interface: 8;
    const Module: 9;
    const Property: 10;
    const Unit: 11;
    const Value: 12;
    const Enum: 13;
    const Keyword: 14;
    const Snippet: 15;
    const Color: 16;
    const File: 17;
    const Reference: 18;
    const Folder: 19;
    const EnumMember: 20;
    const Constant: 21;
    const Struct: 22;
    const Event: 23;
    const Operator: 24;
    const TypeParameter: 25;
}
type CompletionItemKind = 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 20 | 21 | 22 | 23 | 24 | 25;
```

Listing 30: CompletionItemKind enum

QR code



Scan the QR code to access the source code of this thesis:
https://github.com/data-niklas/master_thesis°

Cite this thesis:

BibTex

```bibtex
@thesis{niklas_loeser_2024,
  title = {Interrupt-Guided Neural Code Completion with Static Analysis of Dependencies},
  author = {Niklas Loeser},
  year = {2024},
  month = {9},
  school = {Heidelberg University},
  url = {https://github.com/data-niklas/master_thesis},
}
```